



„Ștefan cel Mare” University of Suceava

*Faculty of Electrical Engineering and
Computer Science*

Universitatii 13, 720229 Suceava, Romania

Email: secretariat@eed.usv.ro, fiesc@usm.ro

ASSIST Software

Tipografiei 1, 720043 Suceava, România

Email: hello@assist.ro

SUMMEREY-I

AI-Powered Study Summarizer

Technical Documentation

Project	Dual-USV
Student	Prelipcean Radu-Adelin
Document version	1.0.7
Document date	March 02 2026

Table of Contents

1. Application Description.....	3
1.1 Application Screenshots.....	4
2. Technologies & Tools	6
3. API Routes	7
4. App Structure	9
5. Database Schema	10
6. Backend App Structure.....	11
7. Git Repository Structure	12
8. App Configuration & Run.....	13
9. Conclusions & Lessons Learned.....	15

Table of Figures

Figure 1 - Landing Page	3
Figure 2 - Guests Dashboard	4
Figure 3 - Personal Documents Archive	4
Figure 4 - "War Rooms" (groups) where users can share their summaries.....	5
Figure 5 - Admin dashboard.....	5
Figure 6 - Custom PDF with the summarized content.....	6
Figure 7 - Swagger UI Docs	9
Figure 8 - Component diagram.....	10

1. Application Description

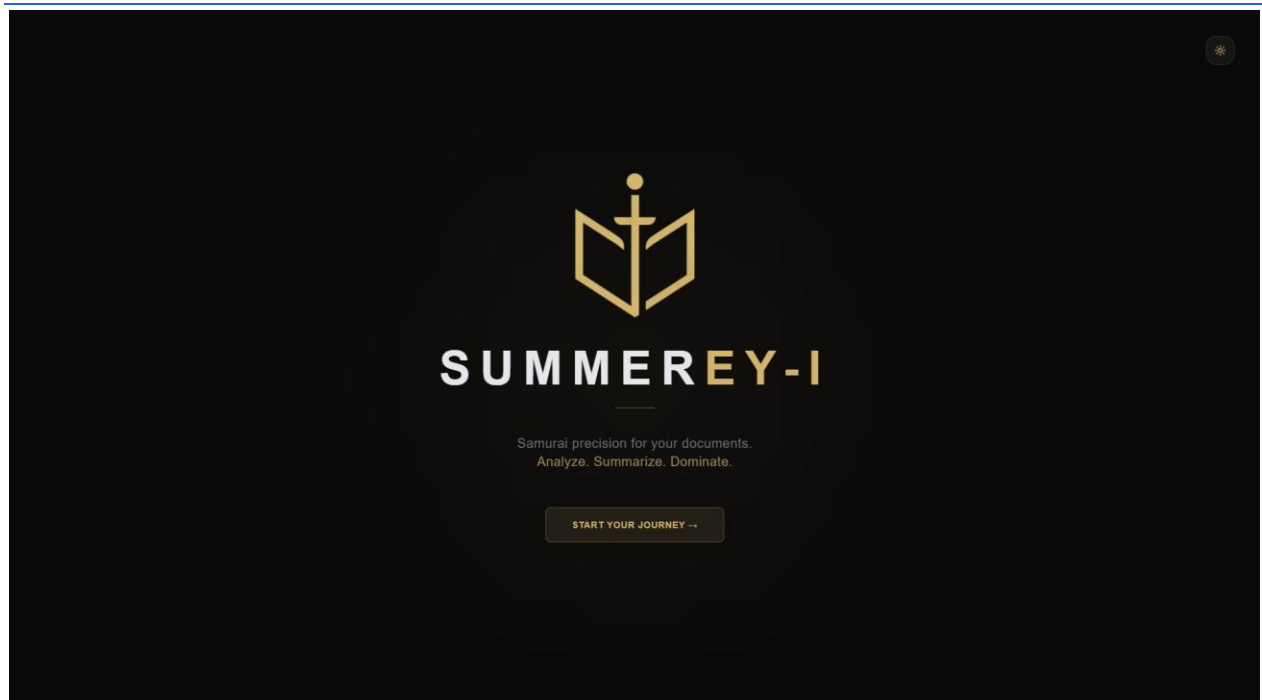


Figure 1 - Landing Page

SUMMEREY-I is a full-stack web application designed to help students and professionals quickly extract key insights from large PDF, DOCX, and TXT documents using artificial intelligence. The application integrates with Google Gemini 2.5 Flash to generate high-quality summaries in the same language as the source document.

Core features:

- **Document upload & AI summarization** — Users upload a document (up to 10 MB) and request either a concise summary (3–5 bullet points) or a detailed one (structured with markdown headings). The AI also performs a safety audit and flags dangerous content.
- **Personal archive** — Registered users maintain a persistent library of all their uploaded documents and summaries, which they can rename, download, or delete.
- **War Rooms** — Collaborative groups where users can share summarized documents with teammates. Each group has a unique 5-character access code for invitations.
- **Guest mode** — Unauthenticated users can summarize up to 2 documents per day (IP-based rate limiting) without creating an account.
- **Admin panel** — Administrators can view all users, ban/unban accounts, review AI-flagged dangerous documents, and verify them as safe.
- **PDF export** — Any summary can be downloaded as a branded PDF with a custom header and footer.

The application follows a **microservice-inspired architecture**: the React frontend is fully decoupled from the FastAPI backend and communicates exclusively via a REST API. Both services are independently deployed and scaled.

1.1 Application Screenshots

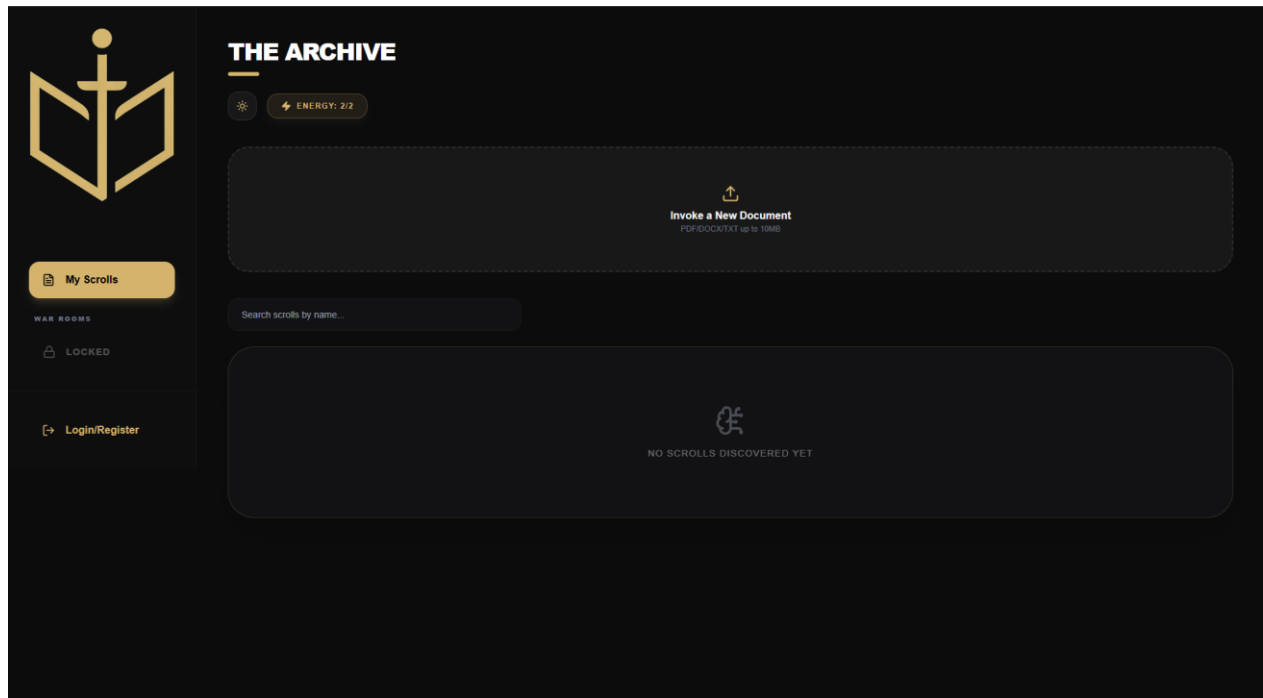


Figure 2 - Guests Dashboard

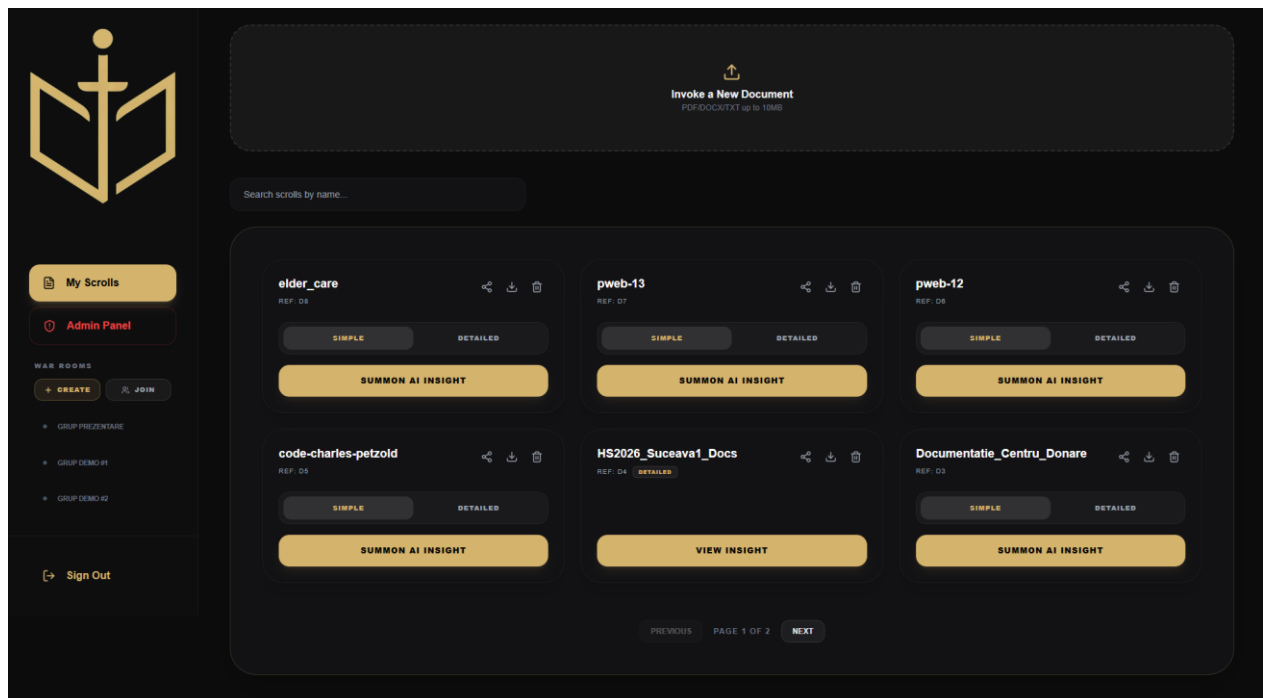


Figure 3 - Personal Documents Archive

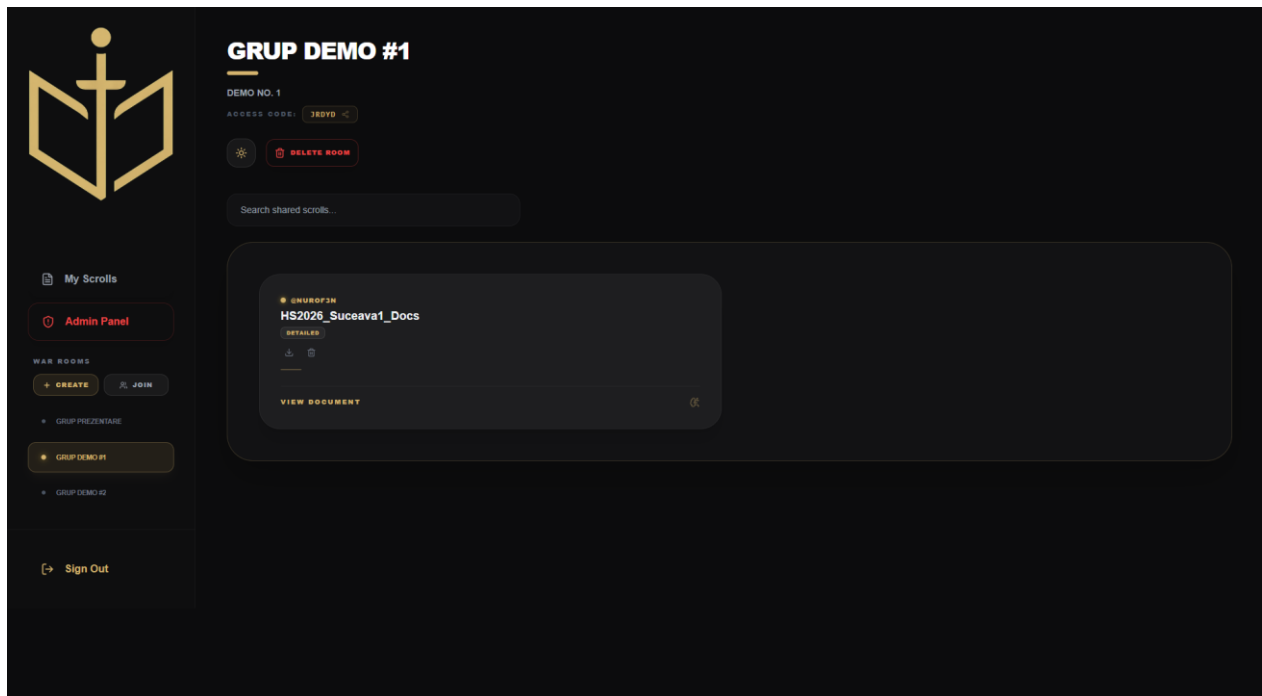


Figure 4 - "War Rooms" (groups) where users can share their summaries

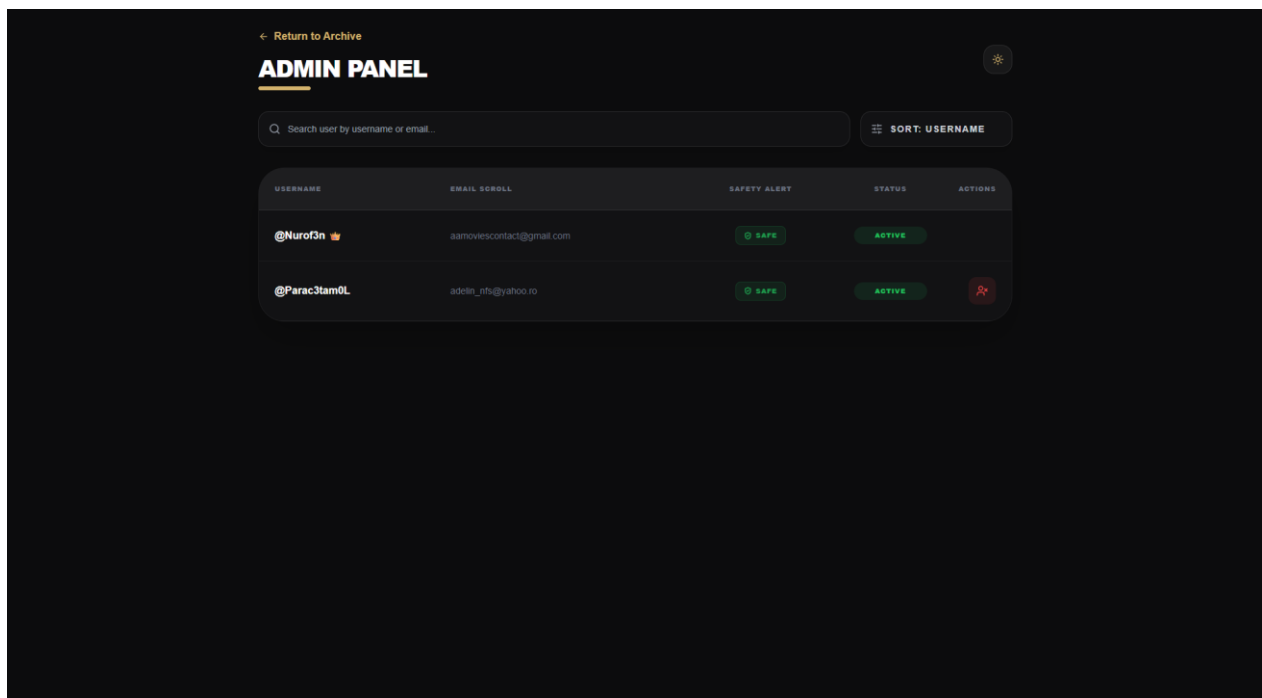


Figure 5 - Admin dashboard

Title

Smart Fall Detection & Vital Monitoring Device for Elderly Care

Authors & Affiliation

Radu-Adelin Prelipcean, Alexandru Rusu, Gheorghe Boca, Cosmin-Mihai Opritescu from the Faculty of Electrical Engineering and Computer Science, "Stefan cel Mare" University of Suceava, Romania.

Abstract & Introduction

This paper introduces a fully autonomous, cloud-independent wearable edge node designed for continuous physiological monitoring and immediate trauma detection in elderly care. It addresses critical limitations of traditional cloud-dependent systems, such as communication dropouts, variable latencies, and data privacy exposures. The proposed device uses an

Figure 6 - Custom PDF with the summarized content

2. Technologies & Tools

Technology	Role
Python 3.12	Backend programming language
FastAPI	REST API framework — provides automatic OpenAPI docs, request validation, dependency injection
SQLAlchemy 2.0	ORM for database models and queries
Alembic	Database schema migration tool
PostgreSQL 15	Relational database — stores users, documents, groups, activity
Google Gemini 2.5 Flash	AI model for generating document summaries
PyJWT / python-jose	JWT token generation and validation for authentication
passlib / bcrypt	Secure password hashing
pypdf / python-docx	Text extraction from PDF and DOCX files

fpdf2	Dynamic PDF generation for summary downloads
React 19	Frontend UI library
Vite	Frontend build tool and dev server
TailwindCSS	Utility-first CSS framework
Axios	HTTP client for API calls
Framer Motion	Animations
React Router v7	Client-side routing
Docker	Containerization of the backend service
GitHub Actions	CI/CD pipeline — runs tests and deploys on every push
Azure App Service	Hosts the backend Docker container
Azure Static Web Apps	Hosts the React frontend (free tier)
Azure Database for PostgreSQL	Managed cloud database
Azure Blob Storage	Cloud file storage for uploaded documents
Azure Container Registry	Private Docker image registry
pytest	Backend automated testing
Vitest + Testing Library	Frontend automated testing

3. API Routes

The full interactive API documentation is available at:

[Swagger UI](#)

Users

- | | |
|----------------------------|--------------------------------------|
| • POST /api/users/register | Register a new user |
| • POST /api/users/login | Authenticate user, returns JWT token |
| • GET /api/users/me | Get current authenticated user |

Documents

- | | |
|-------------------------------------|-------------------------|
| • GET /api/documents/ | List user's documents |
| • POST /api/documents/ | Upload a new document |
| • GET /api/documents/guest-limit | Check guest usage limit |
| • GET /api/documents/{public_id} | Get document details |
| • DELETE /api/documents/{public_id} | Delete a document |

•	PUT	/api/documents/{public_id}/status	Update document status
•	POST	/api/documents/{public_id}/summarize	Generate AI summary
•	PATCH	/api/documents/{public_id}/rename	Rename a document
•	POST	/api/documents/{public_id}/share/{group_id}	Share document to a War Room
•	GET	/api/documents/{public_id}/download-original	Download the original file
•	GET	/api/documents/{public_id}/download-summary	Download summary as PDF

Admin

•	GET	/api/admin/users	Get all users
•	PATCH	/api/admin/users/{user_id}/ban	Toggle user ban
•	POST	/api/admin/users/{user_id}/ban	Ban a user
•	GET	/api/admin/users/{user_id}/dangerous-documents	Get user's flagged documents
•	PATCH	/api/admin/documents/{public_id}/verify-safe	Mark a document as safe

Groups

•	POST	/api/groups/	Create a War Room
•	GET	/api/groups/me	List user's groups
•	POST	/api/groups/{group_id}/members/{user_id}	Invite a user to a group
•	GET	/api/groups/{group_id}/documents	Get War Room documents
•	GET	/api/groups/{group_id}/activity	Get War Room activity feed
•	DELETE	/api/groups/{group_id}	Delete a War Room
•	POST	/api/groups/join/{access_code}	Join a group by access code
•	DELETE	/api/groups/activity/{activity_id}	Remove a shared document
•	POST	/api/groups/leave/{group_id}	Leave a group

default		^
GET	/ Read Root	▼
Users		^
POST	/api/users/register Register	▼
POST	/api/users/login Login	▼
GET	/api/users/me Get Me	🔒 ▼
Documents		^
GET	/api/documents/ Get All Documents Endpoint	🔒 ▼
POST	/api/documents/ Create Document Endpoint	🔒 ▼
GET	/api/documents/guest-limit Get Guest Limit Status	▼
GET	/api/documents/{public_id} Get Document Endpoint	▼
DELETE	/api/documents/{public_id} Delete Document Endpoint	🔒 ▼
PUT	/api/documents/{public_id}/status Update Document Status Endpoint	▼
POST	/api/documents/{public_id}/summarize Summarize Document	🔒 ▼
PATCH	/api/documents/{public_id}/rename Rename Document Endpoint	🔒 ▼
POST	/api/documents/{public_id}/share/{group_id} Share Document	🔒 ▼
GET	/api/documents/{public_id}/download-original Download Original Document	🔒 ▼
GET	/api/documents/{public_id}/download-summary Download Summary Pdf	🔒 ▼

Figure 7 - Swagger UI Docs

4. App Structure

The application consists of two independently deployed services:

study-summarizer/

/— backend/ FastAPI REST API

/— frontend/ React SPA

/— docker-compose.yml Full local development stack

└— .github/workflows/ci.yml CI/CD pipeline

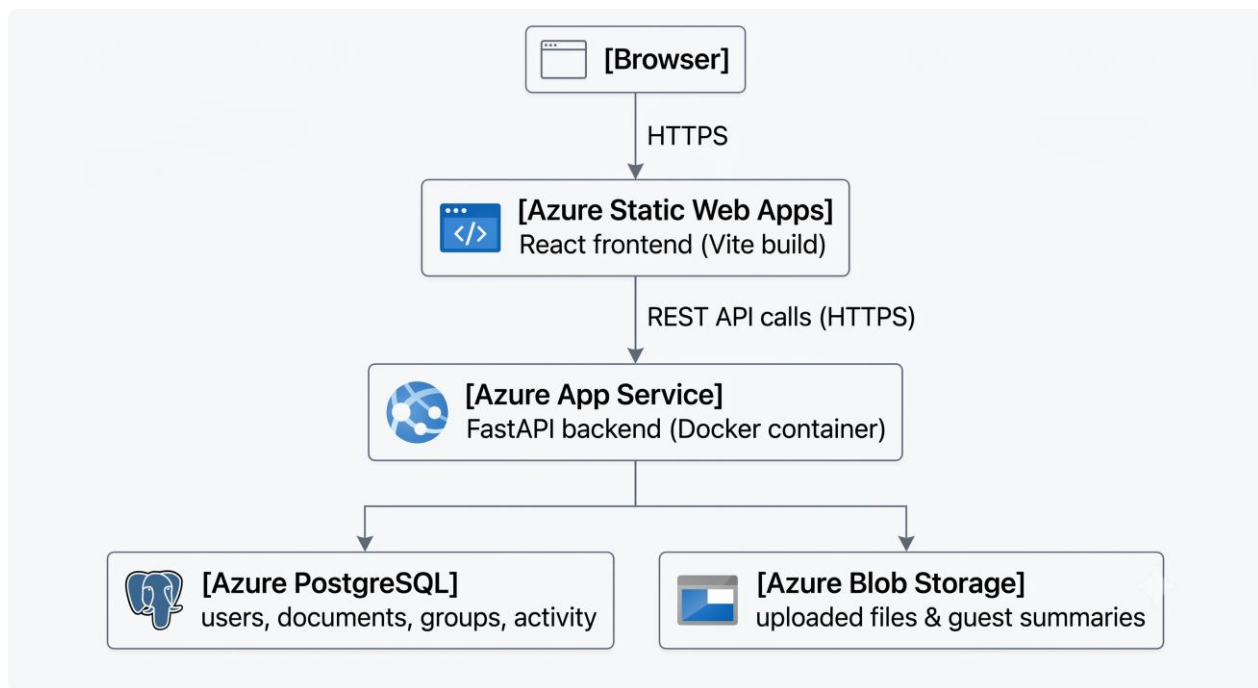


Figure 8 - Component diagram

5. Database Schema

users

id, email (unique), username (unique), hashed_password,
created_at, is_admin, is_banned, ban_reason

documents

id, public_id (unique), owner_id → users, group_id → groups,
title, filename, filetype, uploaded_at, status,
summary (TEXT), summary_type, is_dangerous

groups

id, name (unique), description, access_code (5 chars, unique),
created_at, created_by_id → users
user_group_association (many-to-many junction table)
user_id → users, group_id → groups, role (moderator / member)

group activities

id, group_id → groups, user_id → users,
document_public_id → documents, content, created_at

guest usage

identifier (IP address, PK), count, last_reset

6. Backend App Structure

```
backend/  
├── src/  
│   ├── main.py           FastAPI app entry point, CORS, route registration  
│   ├── core/  
│   │   ├── config.py     Pydantic Settings (environment variables)  
│   │   ├── security.py   JWT creation, bcrypt password hashing  
│   │   ├── storage.py     Storage abstraction (local filesystem ↔ Azure Blob)  
│   │   └── db/  
│   │       └── database.py SQLAlchemy engine and session factory  
│   ├── models/           SQLAlchemy ORM models  
│   │   ├── user.py  
│   │   ├── document.py  
│   │   ├── group.py  
│   │   └── guest_usage.py  
│   ├── api/  
│   │   ├── dependencies.py Auth dependency injection (JWT validation)  
│   │   ├── routers/      HTTP route handlers  
│   │   │   ├── auth_router.py  
│   │   │   ├── document_router.py  
│   │   │   ├── group_router.py  
│   │   │   └── admin_router.py  
│   └── services/         Business logic layer
```

```

├── auth_service.py
├── document_services.py
├── ai_service.py    Google Gemini integration
├── group_service.py
├── admin_service.py
├── repositories/    Database query layer
│   ├── user_repository.py
│   ├── document_repository.py
│   ├── group_repository.py
│   └── guest_usage_repository.py
├── schemas/         Pydantic request/response models
│   ├── user_schemas.py
│   ├── document_schemas.py
│   └── group_schemas.py
├── utils/
│   └── file_parser.py    PDF / DOCX / TXT text extraction
├── tests/           Automated test suite (pytest, 27 tests)
│   ├── conftest.py
│   ├── test_auth.py
│   ├── test_documents.py
│   └── test_groups.py
├── alembic/         Database migration scripts
├── Dockerfile
├── requirements.txt
└── requirements-dev.txt

```

The backend follows a strict **3-layer architecture**:

- **Routers** — validate HTTP input, call services, return responses
- **Services** — business logic and orchestration
- **Repositories** — all database queries isolated here, no business logic

7. Git Repository Structure

```

study-summarizer/
├── .github/
│   └── workflows/
│       └── ci.yml    GitHub Actions: test + build + deploy
├── assets/           Landing page screenshots and logos
├── backend/          FastAPI application
├── frontend/         React application
├── docker-compose.yml    Local dev stack (db + backend + frontend)
└── README.md

```

The project uses a single main branch. Every push to main automatically triggers the full CI/CD pipeline: tests run first, and if they pass, both the backend and frontend are deployed to Azure.

8. App Configuration & Run

The app can be accessed at:

- [Summery-I](#)

or

- [Summery-I - Backup](#)

Prerequisites

- Docker Desktop
- Node.js 20+
- Python 3.12+

Run locally

```
# 1. Clone the repository  
git clone https://github.com/adelinprelipcean/study-summarizer.git  
cd study-summarizer
```

```
# 2. Configure environment  
cp backend/.env.example backend/.env  
# Edit backend/.env — fill in JWT_SECRET and GEMINI_API_KEY
```

```
# 3. Start the full stack  
docker compose up -d --build
```

```
# App is available at:

# Frontend → http://localhost:80

# Backend → http://localhost:8000

# API docs → http://localhost:8000/docs
```

Environment variables

```
DATABASE_URL=postgresql://admin:admin123@db:5432/study_summarizer_db

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_google_ai_key

CORS_ORIGINS=http://localhost:5173

UPLOAD_DIR=uploads

ENV=dev
```

```
# Azure Blob Storage (leave blank to use local filesystem)

AZURE_STORAGE_CONNECTION_STRING=

AZURE_STORAGE_CONTAINER=documents
```

Run tests

```
# Backend — 27 tests

cd backend

pip install -r requirements.txt -r requirements-dev.txt

python -m pytest
```

```
# Frontend — 8 tests

cd frontend

npm install

npm run test
```

9. Conclusions & Lessons Learned

Building SUMMEREY-I was a challenging and highly rewarding project. Starting from scratch over several months, I designed and implemented a complete full-stack application — from the database schema all the way up to a fully automated cloud deployment pipeline.

Technical lessons learned:

- **FastAPI and Python** proved to be an excellent combination for building REST APIs quickly without sacrificing correctness. Automatic request validation, response serialization, and interactive Swagger documentation come out of the box.
- **SQLAlchemy's ORM** allowed complex database relationships — many-to-many groups, document ownership, activity feeds — to be expressed cleanly in Python rather than raw SQL.
- **JWT authentication** taught me how stateless authentication works in practice: how tokens are issued, validated, and how to handle edge cases such as banned users or expired sessions.
- **AI integration** (Google Gemini) demonstrated how to prompt large language models reliably. Getting structured JSON output requires careful prompt engineering and robust fallback handling for cases where the model returns unexpected formats.
- **Cloud deployment on Azure** was the steepest learning curve of the project. Setting up App Service, Container Registry, Static Web Apps, PostgreSQL, and Blob Storage, and connecting them all correctly, required understanding how cloud infrastructure pieces fit together at both the network and authentication levels.
- **CI/CD with GitHub Actions** made the development workflow significantly more professional. Every push to main automatically runs all 35 tests and deploys both services, catching regressions immediately and eliminating manual deployment steps.
- **Automated testing** (pytest + Vitest) taught me the importance of writing tests that are isolated, deterministic, and environment-agnostic — lessons learned through several rounds of debugging tests that passed locally but failed in the CI environment.

Personal conclusions:

This project demonstrated that building a production-ready web application is much more than writing code. It involves architecture decisions, security considerations, database design, error handling, and cloud infrastructure. Each layer of the stack has its own complexity, and making all layers work together reliably is the real challenge of software engineering. The experience of going from a local prototype to a fully deployed, automatically tested cloud application gave me a complete picture of the modern software development lifecycle.